UNIDAD 1

Programación avanzada en SQL – transacciones.

TEMA 1:

Introducción a la administración de bases de datos y transacciones.

Descripción:

En esta unidad, dedicada a la **Programación Avanzada en SQL – Transacciones**, los estudiantes conocerán los fundamentos avanzados de **SQL** aplicados a la gestión de bases de datos, se estudiará el rol del **Administrador** de **Bases de Datos**, el concepto de transacción y las propiedades **ACID**, esenciales para garantizar seguridad e integridad en la información.

Además, se revisarán mecanismos de control de bloqueos, niveles de aislamiento, índices y sentencias avanzadas.

El propósito es que el estudiante adquiera las habilidades necesarias para implementar transacciones seguras y diseñar consultas optimizadas, aplicables a proyectos en Ciencia de Datos e Inteligencia Artificial.

Administración de bases de datos.

PERIODO ACADÉMICO 2026-1S

TABLA DE CONTENIDO

Tema 1. Introducción a la Administración de Bases de Datos y Transacciones..... 2
1.1. Administración de Bases de Datos, Rol y funciones del Administrador de Bases de Datos (DBA). 2
1.2. Concepto de transacción y propiedades ACID..... 3
1.3. Control de bloqueos y Niveles de aislamiento. 6
1.4. Índices y Sentencias Avanzadas. 9
Bibliografía 17

Tema 1. Introducción a la Administración de Bases de Datos y Transacciones.

1.1. Administración de Bases de Datos, Rol y funciones del Administrador de Bases de Datos (DBA).

La administración de bases de datos es la disciplina encargada de planificar, organizar y controlar los datos almacenados en un sistema de gestión, el objetivo principal es garantizar que la información esté disponible, íntegra, segura y accesible para quienes la requieran, incluye tareas como definir estructuras, gestionar usuarios, aplicar medidas de seguridad y optimizar el rendimiento.

Gracias a esta labor, las organizaciones pueden convertir grandes volúmenes de información en un recurso confiable para la toma de decisiones y el desarrollo de aplicaciones tecnológicas (Beynon-Davies, 2021).

Rol y funciones del Administrador de Bases de Datos (DBA).

El Administrador de Bases de Datos (DBA) es el profesional responsable de garantizar que la información almacenada en un sistema de gestión sea segura, íntegra y esté disponible si se requiere, su perfil combina conocimientos técnicos en diseño, configuración y optimización de bases de datos con habilidades para resolver problemas y tomar decisiones oportunas.

Entre sus funciones principales se encuentran la instalación y mantenimiento de gestores, la creación de estructuras de almacenamiento, el control de accesos y permisos, así como la implementación de medidas de seguridad, respaldo y recuperación de datos. Además, debe monitorear el rendimiento, prevenir fallos y proponer mejoras que aseguren la eficiencia del sistema y la continuidad de las operaciones institucionales (Özsu, 2022).

Buenas prácticas en la administración de bases de datos.

La administración de bases de datos requiere seguir buenas prácticas que aseguren la eficiencia y confiabilidad de los sistemas, entre ellas destacan: la implementación de políticas de respaldo y recuperación periódica, el control de accesos mediante usuarios y roles, la actualización constante de parches de seguridad, y el monitoreo del rendimiento para anticipar fallos. También es recomendable documentar procesos, aplicar estándares de normalización y realizar pruebas antes de cambios significativos. Las buenas prácticas fortalecen la seguridad, reducen riesgos y permiten que la información esté siempre disponible para el análisis y la toma de decisiones (Wanigsoriya, 2024).

1.2. Concepto de transacción y propiedades ACID.

Transacción.

Una transacción es una secuencia de operaciones que se ejecutan como una sola unidad indivisible dentro de una base de datos, su objetivo es garantizar que los datos se mantengan consistentes y confiables, aun si ocurren errores o fallos del sistema. Si una parte de la transacción no puede completarse, el sistema revierte todas las operaciones ejecutadas, evita datos incompletos o inconsistentes (Elmasri & Navathe, 2021).

- **Ejemplo:**

  En las operaciones bancarias, una transacción puede consistir en transferir dinero de una cuenta a otra, este proceso involucra dos pasos:

  1. Debitar un monto de la cuenta de origen.
  2. Acreditar el mismo monto en la cuenta de destino.

Si uno de los pasos falla (por ejemplo, se descuenta, pero no se acredita), el sistema anula toda la operación, asegura que no existan pérdidas ni duplicaciones.

Propiedades ACID.

Las transacciones deben cumplir las propiedades **ACID**, consideradas el estándar para garantizar confiabilidad e integridad de una Base de Datos. La **Atomicidad** asegura que todas las operaciones de una transacción se ejecuten por completo o ninguna se realice; la **Consistencia** mantiene reglas y restricciones del sistema, evita estados inválidos; el **Aislamiento** impide que las transacciones interfieran entre sí al momento que se ejecutan en paralelo; y la **Durabilidad** garantiza que los cambios confirmados permanezcan, incluso ante fallos del sistema. Estas propiedades permiten que los sistemas de bases de datos funcionen de forma segura y confiable en entornos multiusuario y son esenciales en aplicaciones críticas como banca, comercio electrónico y gestión de información institucional (Roller, 2024).

- **Ejemplo:**

Código SQL

```sql
BEGIN TRANSACTION;
--Cuenta Debitar
UPDATE cuentas SET saldo = saldo - 500 WHERE id = 1;
--Cuenta Acreditar
UPDATE cuentas SET saldo = saldo + 500 WHERE id = 2;
COMMIT;  -- Termina la transacción
```

Si por alguna razón ocurre un error antes del COMMIT, se revierten todos los cambios de la transacción con un ROLLBACK, preserva la integridad.

Ejemplo:

Un cliente compra un producto, la transacción debe:

1. Descontar el stock del producto.
2. Registrar la venta en la tabla de pedidos.
3. Confirmar el pago en la tabla de transacciones.

Código SQL

```sql
BEGIN TRANSACTION;

-- 1. Descontar el stock
UPDATE productos
SET stock = stock - 1
WHERE producto_id = 101;

-- 2. Registrar la venta
INSERT INTO pedidos (pedido_id, cliente_id, producto_id, fecha)
VALUES (5001, 302, 101, GETDATE());

-- 3. Confirmar el pago
INSERT INTO pagos (pago_id, pedido_id, monto, estado)
VALUES (7001, 5001, 120.00, 'Confirmado');

COMMIT;
```

Si no hay stock disponible o no se realiza el pago el sistema realiza un ROLLBACK.

1.3. Control de bloqueos y Niveles de aislamiento.

Control de bloqueos.

El **bloqueo** es el mecanismo que usan los SGBD para coordinar el acceso concurrente a los datos y evitar inconsistencias (Silberschatz, 2021).

- **Tipos básicos**: **Compartido (S)** para lectura (varias transacciones pueden leer a la vez) y **Exclusivo (X)** para escritura (solo una transacción modifica y las demás esperan).
- **Granularidad**: se puede bloquear **fila**, **página** o **tabla** (a menor granularidad, mayor concurrencia).
- **Objetivo**: prevenir errores como actualizaciones perdidas y estados intermedios visibles.

Ejemplo:

Caso de estudios bloqueo de fila para UPDATE: Dos usuarios intentan descontar stock del mismo producto.

Transacción A

Código SQL

```sql
BEGIN;
-- Lee y bloquea la fila del producto 101 para evitar que otro la modifique
SELECT stock FROM productos WHERE producto_id = 101 FOR UPDATE;

-- Verificación de negocio
UPDATE productos
SET stock = stock - 1
WHERE producto_id = 101;

COMMIT;
```

Transacción B (en paralelo con transacción A)

Código SQL

```
BEGIN;
-- Esta instrucción esperará hasta que A haga COMMIT/ROLLBACK
SELECT stock FROM productos WHERE producto_id = 101 FOR UPDATE;

UPDATE productos
SET stock = stock - 1
WHERE producto_id = 101;

COMMIT;
```

Con el uso de los bloqueos se puede garantizar:

- **Atomicidad/consistencia:** no hay doble descuento; la segunda transacción espera el resultado de la primera (Barthels et al., 2019).
- **Aislamiento:** nadie puede modificar esa fila mientras está bloqueada (Barthels et al., 2019).

Niveles de aislamiento.

Los niveles de aislamiento definen el grado en que una transacción está protegida de los efectos de otras transacciones concurrentes (Silberschatz et al., 2020), de acuerdo con el estándar SQL, existen cuatro niveles principales:

- **Read Uncommitted**, pueden ocurrir lecturas de datos de transacciones no confirmadas
- **Read Committed**, evita lecturas de datos de transacciones no confirmadas, pero permite lecturas no repetibles
- **Repeatable Read**, que asegura consistencia en las lecturas, aunque pueden existir anomalías con inserciones

- **Serializable**, el nivel más estricto, que garantiza un comportamiento equivalente a la ejecución secuencial.

Elegir el nivel adecuado implica equilibrar entre rendimiento y seguridad de los datos, depende de la aplicación y su criticidad.

Ejemplo:

Aislamiento de una transacción bancaria

Código SQL

```sql
-- Establecer el nivel más estricto
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN TRANSACTION;

-- Consulta de cuentas con saldo mayor a 1000
SELECT * FROM cuentas WHERE saldo > 1000;

-- Transferencia protegida contra inconsistencias
UPDATE cuentas SET saldo = saldo - 200 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 200 WHERE id = 2;

COMMIT;
```

Con los niveles de aislamiento en este ejemplo se garantiza, que ninguna otra transacción puede insertar, modificar o leer registros que alteren el resultado hasta que termine el COMMIT, evita lecturas de datos de transacciones no confirmadas, lectura no repetible y anomalías de inserción (Bouajjani et al., 2025).

1.4. Índices y Sentencias Avanzadas.

Índices.

Un **índice** es una estructura de acceso adicional asociada a una tabla o vista que mantiene **claves ordenadas** derivadas de una o varias columnas, junto con **punteros** a las filas donde se encuentran los datos, su propósito es **acelerar** búsquedas, uniones (*JOIN*), ordenamientos (*ORDER BY*) y agregaciones (*GROUP BY*) (Silberschatz, 2021).

En la mayoría de SGBD se implementa con una estructura (árbol B); según el caso, también existen hash, GiST/GIN o BRIN. Los índices mejoran las consultas, pero añaden costo a INSERT/UPDATE/DELETE y consumen espacio; por ello deben diseñarse tomando en cuenta selectividad, cardinalidad y patrones de consulta (Silberschatz, 2021).

Índices agrupados (clustered).

Un índice agrupado determina el orden físico de almacenamiento de los datos en una tabla, de manera similar a un diccionario ordenado por letras; cada tabla puede tener solo un índice agrupado, ya que define cómo se organizan sus filas en disco. Este tipo de índice es especialmente útil para consultas que devuelven rangos de valores, ya que, al estar los datos ordenados, la búsqueda y recuperación es más rápida. Sin embargo, las operaciones de inserción o actualización en columnas clave pueden requerir más recursos, pues pueden reorganizar el almacenamiento de la tabla (Silberschatz, 2021).

Crear un índice agrupado (Clustered)
Un índice agrupado ordena físicamente los datos de la tabla según la columna especificada.

Código SQL
```
-- Crear índice agrupado sobre la columna id_pedido
CREATE CLUSTERED INDEX IX_Pedidos_Id
ON Pedidos (id_pedido);
```

En este caso, la tabla Pedidos se organizará en disco según id_pedido. Generalmente, las claves primarias se definen automáticamente como índices agrupados.

Índices no agrupados (non-clustered).

Un índice no agrupado crea una estructura separada de la tabla, que almacena las claves ordenadas y punteros a las filas correspondientes, a diferencia del índice agrupado, no altera el orden físico de los datos. Una tabla puede tener varios índices no agrupados, lo que permite optimizar diferentes consultas según las necesidades del sistema, estos índices son ideales para búsquedas puntuales o para consultas que filtran por condiciones específicas en columnas no ordenadas físicamente. El uso excesivo puede ralentizar las operaciones de escritura, ya que cada modificación de datos debe actualizar los índices asociados (Silberschatz, 2021).

Crear un índice no agrupado (Non-Clustered)

Un índice no agrupado genera una estructura aparte que contiene las claves ordenadas y punteros a las filas

Código SQL
```
-- Crear índice no agrupado sobre la columna cliente_id
CREATE NONCLUSTERED INDEX IX_Pedidos_Cliente
ON Pedidos (cliente_id);
```

Este índice optimiza consultas como:

Código SQL
```sql
SELECT *
FROM Pedidos
WHERE cliente_id = 202;
```

Sentencias Avanzadas.

Las sentencias avanzadas en SQL facilitan la construcción de operaciones complejas sobre múltiples tablas y grandes volúmenes de datos, estas sentencias incluyen mecanismos para combinar información, realizar cálculos intermedios, definir resultados temporales o analizar patrones dentro de los datos. Es importante el uso de estas sentencias, ya que optimiza la recuperación de información y fortalece las capacidades de análisis en las aplicaciones (Silberschatz, 2021).

JOIN.

Los **JOIN** son sentencias avanzadas que permiten combinar datos de dos o más tablas relacionadas a través de una columna común. Existen varios tipos:

- **INNER JOIN**: devuelve únicamente los registros que tienen coincidencias en ambas tablas.
- **LEFT JOIN**: devuelve todos los registros de la tabla izquierda y los coincidentes de la derecha.
- **RIGHT JOIN**: devuelve todos los registros de la tabla derecha y los coincidentes de la izquierda.
- **FULL JOIN**: devuelve todos los registros de ambas tablas, coincidan o no.

Los JOIN son fundamentales en bases de datos normalizadas, ya que facilitan consultas sobre múltiples entidades sin redundancia de información (Shin et al., 2022).

- **Ejemplo:**

(INNER JOIN y LEFT JOIN)

Código SQL

```
-- INNER JOIN: clientes que tienen pedidos
SELECT c.nombre, p.id_pedido, p.monto
FROM clientes c
INNER JOIN pedidos p
  ON c.cliente_id = p.cliente_id;

-- LEFT JOIN: lista todos los clientes, incluso los que no tienen
pedidos
SELECT c.nombre, p.id_pedido, p.monto
FROM clientes c
LEFT JOIN pedidos p
  ON c.cliente_id = p.cliente_id;
```

En el ejemplo, el **INNER JOIN** muestra únicamente clientes con pedidos registrados, mientras que el **LEFT JOIN** incluye también a los clientes sin pedidos, devuelve NULL en las columnas de pedidos.

Subconsultas.

Una **subconsulta** es una instrucción SQL anidada dentro de otra consulta, su función es devolver un conjunto de resultados que se utilizan como condición o fuente de datos para la consulta principal. Las subconsultas pueden aparecer en cláusulas como WHERE, FROM o SELECT, y permiten resolver problemas complejos de forma estructurada (Silberschatz, 2021).

Existen dos tipos principales:

- **Subconsulta no correlacionada:** Se ejecuta una sola vez y su resultado se reutiliza en la consulta externa.
- **Subconsulta correlacionada:** Se evalúa repetidamente, una vez por cada fila procesada en la consulta externa.

Ejemplo:

Subconsulta no correlacionada: Identificar clientes con compras superiores al gasto promedio.

Código SQL

```sql
SELECT nombre, cliente_id
FROM clientes
WHERE cliente_id IN (
    SELECT cliente_id
    FROM pedidos
    GROUP BY cliente_id
    HAVING SUM(monto) > (
        SELECT AVG(monto) FROM pedidos
    )
);
```

En el ejemplo, las subconsultas calculan el promedio de compras una sola vez y comparan con cada cliente.

Ejemplo:

Subconsulta correlacionada: Listar clientes y sus pedidos que superan el promedio individual de ese cliente.

Código SQL

```sql
SELECT c.nombre, p.id_pedido, p.monto
FROM clientes c
JOIN pedidos p ON c.cliente_id = p.cliente_id
WHERE p.monto > (
    SELECT AVG(p2.monto)
    FROM pedidos p2
    WHERE p2.cliente_id = c.cliente_id
);
```


ACREDITADA QS World University Rankings Latin America THE World University Rankings 2023 Latin America THE Impact Summit soludable

En el ejemplo, la subconsulta se vuelve a ejecutar por cada cliente, calcula su promedio individual y compara cada pedido con él.

Common Table Expressions (CTE).

Las **Common Table Expressions (CTE)** son estructuras temporales que permiten nombrar y reutilizar el resultado de una consulta dentro de una instrucción SQL más grande, se definen con la cláusula WITH y son especialmente útiles para organizar consultas complejas, evita anidaciones difíciles de leer. Una CTE mejora la claridad del código, facilita el mantenimiento y puede usarse múltiples veces dentro de la misma consulta (Stuparu, & Petrescu, 2025).

- **Ejemplo:**

Código SQL

```sql
WITH compras_cliente AS (
    SELECT cliente_id, SUM(monto) AS total
    FROM pedidos
    GROUP BY cliente_id
)
SELECT c.nombre, cc.total
FROM clientes c
JOIN compras_cliente cc
  ON c.cliente_id = cc.cliente_id
WHERE cc.total > 1000;
```

La CTE compras_cliente calcula los totales por cliente y luego se usa en la consulta principal para filtrar aquellos con más de 1000 en compras.

Consultas recursivas.

Las consultas recursivas son un tipo especial de sentencia que permiten recorrer datos organizados en estructuras jerárquicas o repetitivas, se basan en el uso de CTE recursivas con la cláusula WITH RECURSIVE, donde

una parte de la consulta inicializa los datos base y otra parte los expande sucesivamente hasta cumplir una condición de terminación (Stuparu, & Petrescu, 2025). Este enfoque es muy útil en problemas como organigramas de empresas, árboles genealógicos, clasificación de productos o gestión de rutas en redes, la recursividad evita múltiples uniones manuales y permite un manejo más flexible de jerarquías.

- **Ejemplo:**

Caso de estudio: Jerarquía de empleados

Código SQL

```sql
WITH RECURSIVE jerarquia AS (
    -- Caso base: directores (no tienen jefe)
    SELECT empleado_id, nombre, jefe_id, 1 AS nivel
    FROM empleados
    WHERE jefe_id IS NULL

    UNION ALL

    -- Paso recursivo: empleados con jefe
    SELECT e.empleado_id, e.nombre, e.jefe_id, j.nivel + 1
    FROM empleados e
    JOIN jerarquia j ON e.jefe_id = j.empleado_id
)
SELECT *
FROM jerarquia
ORDER BY nivel, nombre;
```

En el ejemplo, la consulta comienza con los directores (nivel 1) y luego, de forma recursiva, agrega a los empleados subordinados, incrementa el nivel jerárquico en cada paso.

Consultas con funciones de agregación y de ventana.

Las **funciones de agregación** (SUM, AVG, COUNT, MIN, MAX) permiten obtener resultados globales o por grupos de datos, lo que las hace esenciales para el análisis de información en SQL, sin embargo, al agrupar, se pierde el detalle fila por fila. Para resolver esta limitación existen las **funciones de ventana (window functions)**, que aplican cálculos sobre un conjunto de filas relacionadas, sin colapsar los resultados individuales (Leis et al., 2020). Para esto de usa la cláusula OVER para definir la "ventana" de análisis, que puede dividirse en particiones (PARTITION BY) y ordenarse (ORDER BY), estas funciones resultan muy útiles en reportes, análisis estadístico y aplicaciones de Ciencia de Datos.

- **Ejemplo:**

Código SQL

```sql
-- Total de compras por cliente (agregación)
SELECT cliente_id, SUM(monto) AS total
FROM pedidos
GROUP BY cliente_id;

-- Ranking de clientes por monto total (ventana)
SELECT cliente_id,
       SUM(monto) AS total,
       RANK() OVER (ORDER BY SUM(monto) DESC) AS posicion
FROM pedidos
GROUP BY cliente_id;

-- Promedio de compras de cada cliente comparado con su historial
SELECT cliente_id, monto,
       AVG(monto) OVER (PARTITION BY cliente_id) AS promedio_cliente
FROM pedidos;
```

En el ejemplo, el primer grupo de instrucciones usa una agregación clásica, el segundo aplica RANK() para asignar una posición a cada cliente según sus compras y el tercero calcula el promedio por cliente, pero sin perder el detalle de cada pedido.

Bibliografía

Barthels, C., Müller, I., Taranov, K., Alonso, G., & Hoefler, T. (2019). Strong consistency is not hard to get: Two-phase locking and two-phase commit on thousands of cores. *Proceedings of the VLDB Endowment*, *12*(13), 2325-2338. https://doi.org/10.14778/3358701.3358702

Beynon-Davies, P. (2021). Data and Database Administration. En P. Beynon-Davies (Ed.), *Database Systems* (pp. 129-143). Macmillan Education UK. https://doi.org/10.1007/978-1-349-13722-0_11

Bouajjani, A., Enea, C., & Román-Calvo, E. (2025). On the Complexity of Checking Mixed Isolation Levels for SQL Transactions. En R. Piskac & Z. Rakamarić (Eds.), *Computer Aided Verification* (Vol. 15934, pp. 315-337). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-98685-7_15

Connolly, T. M., & Begg, C. E. (2021). *Database systems: A practical approach to design, implementation, and management* (4th ed). Addison-Wesley.

Elmasri, R., & Navathe, S. (2021). *Fundamentals of database systems* (Seventh edition). Pearson.

Foufoulas, Y., & Simitsis, A. (2023). Efficient Execution of User-Defined Functions in SQL Queries. *Proceedings of the VLDB Endowment*, *16*(12), 3874-3877. https://doi.org/10.14778/3611540.3611574

Gupta, S., & Ramachandra, K. (2021). Procedural extensions of SQL: Understanding their usage in the wild. *Proceedings of the VLDB Endowment*, *14*(8), 1378-1391. https://doi.org/10.14778/3457390.3457402

Jorge Santiago Nolasco Valenzuela. (2018). *Python Aplicaciones prácticas*. RA-MA Editorial.

Leis, V., Kundhikanjana, K., Kemper, A., & Neumann, T. (2020). Efficient processing of window functions in analytical SQL queries. *Proc. VLDB Endow.*, *8*(10), 1058-1069. https://doi.org/10.14778/2794367.2794375

Özsu, M. T. (2022). Database Administrator (DBA). En *Encyclopedia of Database Systems* (pp. 935-936). Springer, New York, NY. https://doi.org/10.1007/978-1-4614-8265-9_80652

Rahman, M. M., Islam, S., Kamruzzaman, M., & Joy, Z. H. (2024). Advanced query optimization in SQL databases for real-time big data analytics. *Academic Journal on Business Administration, Innovation & Sustainability*, *4*(3), 1-14.

Rodríguez Guerrero, R., Vanegas, C. A., & Castang Montiel, G. (2020). *Python a su alcance*. Editorial Universidad Distrital Francisco José de Caldas. Editorial UD.

Roller, J. (2024, julio 8). Ensuring Data Integrity: The Role of ACID Transactions in Modern Distributed Microservices Architecture. *IEEE Computer Society*. https://www.computer.org/publications/tech-news/community-voices/acid-transactions-in-distributed-microservices-architecture/

Shin, Y., Ahn, J., & Im, D.-H. (2022). Join optimization for inverted index technique on relational database management systems. *Expert Systems with Applications*, *198*, 116956. https://doi.org/10.1016/j.eswa.2022.116956

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (5th ed). McGraw-Hill Higher Education.

Silberschatz, A. (with Korth, H. F., Sudarshan, S., Sáenz Pérez, F., García Cordero, A., Correas Fernández, J., & Grau Fernández, L.). (2021). *Fundamentos de bases de datos* (5a ed). McGraw-Hill.

Stuparu, D., & Petrescu, M. (2025). Common Table Expression: Different database systems approach. *ResearchGate*. https://www.researchgate.net/publication/259441876_Common_Table_Expression_Different_database_systems_approach

Wanigsoriya, S. (2024). Database Administration Developments and Research in Contemporary Digital Environments. *ResearchGate*. https://doi.org/10.13140/RG.2.2.25897.81763